

20 Exercícios Comentados de C

Do básico ao exercício integrado — variáveis, entrada de dados, condicionais, laços de repetição, vetores, strings e structs.

Como estudar: tente resolver cada exercício sozinho antes de olhar a solução. Depois, compare seu código com a solução comentada e compile o programa no seu ambiente C.

1. Variáveis e operações básicas

Enunciado: Crie um programa que receba o nome de uma pessoa, sua idade e sua altura. Depois, mostre esses dados e calcule quantos anos ela terá daqui a 10 anos.

Solução:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char nome[100];
    int idade;
    float altura;

    // Lê o nome. fgets permite nomes com espaços.
    printf("Nome: ");
    fgets(nome, sizeof(nome), stdin);
    nome[strcspn(nome, "\n")] = '\\0';

    // Lê a idade e a altura.
    printf("Idade: ");
    scanf("%d", &idade);

    printf("Altura (m): ");
    scanf("%f", &altura);

    // Mostra os dados e calcula a idade futura.
    printf("\\nNome: %s\\n", nome);
    printf("Idade: %d\\n", idade);
    printf("Altura: %.2f m\\n", altura);
    printf("Daqui a 10 anos: %d\\n", idade + 10);

    return 0;
}
```

Comentário: O scanf precisa receber o endereço da variável com &, para poder armazenar nela o valor digitado. O tipo char[] é usado para representar uma string em C; diferentemente de Python, C não possui um tipo string nativo.

2. Entrada de dados e média

Enunciado: Leia três notas de um aluno e calcule a média aritmética. Mostre a média com duas casas decimais.

Solução:

```
#include <stdio.h>

int main(void) {
    float nota1, nota2, nota3, media;

    // Lê as três notas.
    printf("Nota 1: ");
    scanf("%f", &nota1);

    printf("Nota 2: ");
    scanf("%f", &nota2);

    printf("Nota 3: ");
    scanf("%f", &nota3);
```

```

// Calcula a média.
media = (nota1 + nota2 + nota3) / 3.0f;

// %.2f mostra exatamente duas casas decimais.
printf("Media: %.2f\\n", media);

return 0;
}

```

Comentário: Em C, float é usado para números reais. O especificador %.2f controla a apresentação para duas casas decimais.

3. Condicional simples — número positivo

Enunciado: Leia um número e informe somente se ele é positivo. Se não for positivo, o programa pode não imprimir nada.

Solução:

```

#include <stdio.h>

int main(void) {
    float numero;

    printf("Digite um numero: ");
    scanf("%f", &numero);

    // O bloco do if só executa quando a condição é verdadeira.
    if (numero > 0) {
        printf("O numero e positivo.\\n");
    }

    return 0;
}

```

Comentário: O if em C funciona de maneira semelhante ao if de Python, mas o bloco de comandos é delimitado por chaves { }.

4. if, else if e else — situação do aluno

Enunciado: Leia a média de um aluno. Mostre “Aprovado” para média maior ou igual a 6, “Recuperação” para média entre 4 e 5,99 e “Reprovado” para média menor que 4.

Solução:

```

#include <stdio.h>

int main(void) {
    float media;

    printf("Media: ");
    scanf("%f", &media);

    // Testa as possibilidades em ordem.
    if (media >= 6) {
        printf("Aprovado\\n");
    } else if (media >= 4) {
        printf("Recuperacao\\n");
    } else {
        printf("Reprovado\\n");
    }

    return 0;
}

```

Comentário: Em C, o equivalente ao elif de Python é else if. O else é executado quando nenhuma condição anterior foi satisfeita.

5. Condicionais aninhadas — classificação de idade

Enunciado: Leia a idade de uma pessoa. Se ela tiver 18 anos ou mais, diga que é adulta e, dentro desse caso, informe se tem 60 anos ou mais. Para menores de 18, diga que é menor de idade.

Solução:

```
#include <stdio.h>

int main(void) {
    int idade;

    printf("Idade: ");
    scanf("%d", &idade);

    // Primeira decisão: verificar se é maior de idade.
    if (idade >= 18) {
        printf("Maior de idade.\n");

        // Segunda decisão dentro da primeira.
        if (idade >= 60) {
            printf("Tambem esta na faixa de 60 anos ou mais.\n");
        }
    } else {
        printf("Menor de idade.\n");
    }

    return 0;
}
```

Comentário: O segundo if está aninhado dentro do primeiro. Assim, a segunda verificação só acontece quando idade >= 18.

6. for simples — tabuada

Enunciado: Leia um número inteiro e mostre sua tabuada de 1 a 10 usando for.

Solução:

```
#include <stdio.h>

int main(void) {
    int numero, i;

    printf("Numero: ");
    scanf("%d", &numero);

    // O for começa em 1, continua até 10 e incrementa i de 1 em 1.
    for (i = 1; i <= 10; i++) {
        printf("%d x %d = %d\\n", numero, i, numero * i);
    }

    return 0;
}
```

Comentário: O for de C concentra inicialização, condição e atualização na mesma linha: for (início; condição; incremento).

7. while simples — senha

Enunciado: Peça uma senha ao usuário. Enquanto a senha estiver incorreta, continue pedindo. Quando estiver correta, mostre uma mensagem de acesso liberado.

Solução:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char senha[50];
    const char senha_correta[] = "python123";

    // Continua pedindo enquanto a senha estiver errada.
    do {
        printf("Digite a senha: ");
        scanf("%49s", senha);

        if (strcmp(senha, senha_correta) != 0) {
            printf("Senha incorreta.\\n");
        }
    } while (strcmp(senha, senha_correta) != 0);

    printf("Acesso liberado!\\n");

    return 0;
}
```

Comentário: Como strings em C são vetores de char, usamos strcmp() para compará-las. Aqui foi usado do...while para garantir pelo menos uma leitura da senha.

8. for aninhado — matriz de números

Enunciado: Mostre uma matriz 3 × 4 contendo os números de 1 a 12, usando um for dentro de outro for.

Solução:

```
#include <stdio.h>

int main(void) {
    int linha, coluna;
    int numero = 1;

    // O for externo controla as 3 linhas.
```

```

for (linha = 0; linha < 3; linha++) {

    // O for interno controla as 4 colunas.
    for (coluna = 0; coluna < 4; coluna++) {
        printf("%2d ", numero);
        numero++;
    }

    // Quebra a linha depois de cada conjunto de 4 números.
    printf("\n");
}

return 0;
}

```

Comentário: O for externo repete três vezes e, para cada linha, o for interno repete quatro vezes. Essa estrutura é muito usada para percorrer matrizes.

9. while aninhado — contador de combinações

Enunciado: Use dois while aninhados para imprimir todas as combinações de dois números, sendo o primeiro de 1 a 3 e o segundo de 1 a 2.

Solução:

```

#include <stdio.h>

int main(void) {
    int a = 1;
    int b;

    // Primeiro while controla o primeiro número.
    while (a <= 3) {
        b = 1;

        // Para cada valor de a, b começa novamente em 1.
        while (b <= 2) {
            printf("(%d, %d)\n", a, b);
            b++;
        }

        a++;
    }

    return 0;
}

```

Comentário: É necessário atualizar a e b para evitar um laço infinito. O while interno é reiniciado para cada novo valor de a.

10. Vetor — adicionar, remover e percorrer

Enunciado: Crie um vetor de frutas. Como C não possui listas dinâmicas nativas como Python, use um vetor de strings e simule a remoção de um item deslocando os elementos.

Solução:

```

#include <stdio.h>
#include <string.h>

int main(void) {
    char frutas[10][30] = {"maca", "banana", "laranja"};
    int quantidade = 3;
    int i, posicao = 1;

    // "Adiciona" uva na próxima posição disponível.
    strcpy(frutas[quantidade], "uva");
    quantidade++;

    // Remove banana deslocando os elementos seguintes uma posição.
    for (i = posicao; i < quantidade - 1; i++) {
        strcpy(frutas[i], frutas[i + 1]);
    }
}

```

```
}
quantidade--;

// Percorre o vetor e mostra as frutas restantes.
for (i = 0; i < quantidade; i++) {
    printf("%s\\n", frutas[i]);
}

return 0;
}
```

Comentário: Em C, um vetor tem tamanho definido. Para reproduzir operações parecidas com `append()` e `remove()`, controlamos manualmente a quantidade de elementos e deslocamos os dados quando necessário.

11. Vetor — notas e maior valor

Enunciado: Leia cinco notas, coloque-as em um vetor e depois mostre todas as notas, a maior, a menor e a média.

Solução:

```
#include <stdio.h>

int main(void) {
    float notas[5];
    float maior, menor, soma = 0;
    int i;

    // Lê as cinco notas e calcula a soma.
    for (i = 0; i < 5; i++) {
        printf("Digite a nota %d: ", i + 1);
        scanf("%f", &notas[i]);
        soma += notas[i];
    }

    // Inicializa maior e menor com o primeiro elemento.
    maior = notas[0];
    menor = notas[0];

    // Procura os valores extremos.
    for (i = 1; i < 5; i++) {
        if (notas[i] > maior)
            maior = notas[i];

        if (notas[i] < menor)
            menor = notas[i];
    }

    printf("\nNotas: ");
    for (i = 0; i < 5; i++)
        printf("%.2f ", notas[i]);

    printf("\nMaior: %.2f\n", maior);
    printf("Menor: %.2f\n", menor);
    printf("Media: %.2f\n", soma / 5.0f);

    return 0;
}
```

Comentário: Como C não possui max(), min() e sum() para vetores de forma equivalente ao Python, percorremos o vetor com laços e calculamos essas informações.

12. Dados fixos — meses do ano

Enunciado: Crie um vetor de strings contendo os meses do ano. Mostre o terceiro mês e percorra todos os meses usando for.

Solução:

```
#include <stdio.h>

int main(void) {
    const char *meses[] = {
        "janeiro", "fevereiro", "marco", "abril",
        "maio", "junho", "julho", "agosto",
        "setembro", "outubro", "novembro", "dezembro"
    };

    int i;

    // Os índices começam em zero: índice 2 é o terceiro mês.
    printf("Terceiro mes: %s\n\n", meses[2]);

    // Percorre os 12 elementos do vetor.
    for (i = 0; i < 12; i++) {
        printf("%s\n", meses[i]);
    }
}
```

```

    }

    return 0;
}

```

Comentário: C não possui tuplas como Python. Para dados fixos e ordenados, um vetor de strings declarado como const é uma alternativa adequada. Assim como em Python, o índice do primeiro elemento é 0.

13. struct — adicionar, alterar e remover dados

Enunciado: Como C não possui dicionários nativos, use uma struct para representar uma pessoa. Crie nome e idade, adicione uma profissão, altere a idade e depois remova a profissão simulando a remoção com um campo de controle.

Solução:

```

#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[50];
    int idade;
    char profissao[50];
    int tem_profissao;
} Pessoa;

int main(void) {
    Pessoa pessoa = {"Carlos", 30, "", 0};

    // Adiciona a profissão.
    strcpy(pessoa.profissao, "Químico");
    pessoa.tem_profissao = 1;

    // Altera a idade.
    pessoa.idade = 31;

    // "Remove" a profissão limpando o campo e desativando-o.
    pessoa.profissao[0] = '\\0';
    pessoa.tem_profissao = 0;

    // Mostra os dados existentes.
    printf("Nome: %s\\n", pessoa.nome);
    printf("Idade: %d\\n", pessoa.idade);

    if (pessoa.tem_profissao)
        printf("Profissao: %s\\n", pessoa.profissao);
    else
        printf("Profissao: nao cadastrada\\n");

    return 0;
}

```

Comentário: Neste exercício, struct ocupa o papel de agrupar campos relacionados. Não existe um equivalente direto ao dicionário Python; para aplicações maiores, estruturas como arrays de structs podem cumprir função semelhante.

14. Vetor de structs — cadastro de produtos

Enunciado: Crie um vetor de produtos contendo nome e preço. Permita ao usuário digitar o nome de um produto para consultar seu preço. Trate o caso em que o produto não existe.

Solução:

```

#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[30];
    float preco;
} Produto;

```

```

int main(void) {
    Produto produtos[] = {
        {"arroz", 25.90f},
        {"feijao", 8.50f},
        {"cafe", 18.00f}
    };

    char busca[30];
    int i, encontrado = 0;

    printf("Produto: ");
    scanf("%29s", busca);

    // Percorre os produtos procurando uma correspondência.
    for (i = 0; i < 3; i++) {
        if (strcmp(busca, produtos[i].nome) == 0) {
            printf("Preco: R$ %.2f\\n", produtos[i].preco);
            encontrado = 1;
            break;
        }
    }

    // Se nenhum produto foi encontrado, informa ao usuário.
    if (!encontrado)
        printf("Produto nao cadastrado.\\n");

    return 0;
}

```

Comentário: Em vez de testar se uma chave existe com in, como em Python, C percorre o vetor e compara as strings com strcmp(). A variável encontrado registra se houve correspondência.

15. Vetores + condicionais — separar pares e ímpares

Enunciado: Leia dez números inteiros e coloque os pares em um vetor e os ímpares em outro. No final, mostre os dois vetores.

Solução:

```

#include <stdio.h>

int main(void) {
    int pares[10], impares[10];
    int qtd_pares = 0, qtd_impares = 0;
    int numero, i;

    // Lê dez números e separa usando o resto da divisão.
    for (i = 0; i < 10; i++) {
        printf("Numero %d: ", i + 1);
        scanf("%d", &numero);

        if (numero % 2 == 0) {
            pares[qtd_pares] = numero;
            qtd_pares++;
        } else {
            impares[qtd_impares] = numero;
            qtd_impares++;
        }
    }

    printf("\\n\\nPares: ");
    for (i = 0; i < qtd_pares; i++)
        printf("%d ", pares[i]);

    printf("\\n\\nImpares: ");
    for (i = 0; i < qtd_impares; i++)
        printf("%d ", impares[i]);

    printf("\\n\\n");
    return 0;
}

```

Comentário: O operador % calcula o resto da divisão inteira. Se `numero % 2` for 0, o número é par. As variáveis `qtd_pares` e `qtd_impares` controlam quantos elementos foram armazenados em cada vetor.

16. Fibonacci com for

Enunciado: Leia a quantidade de termos desejada e mostre a sequência de Fibonacci usando for. Considere que a sequência começa em 0, 1, 1, 2...

Solução:

```
#include <stdio.h>

int main(void) {
    int n, i;
    long long a = 0, b = 1, proximo;

    printf("Quantidade de termos: ");
    scanf("%d", &n);

    // Repete exatamente n vezes.
    for (i = 0; i < n; i++) {
        printf("%lld ", a);

        // Calcula o próximo termo e atualiza as variáveis.
        proximo = a + b;
        a = b;
        b = proximo;
    }

    printf("\n");
    return 0;
}
```

Comentário: As variáveis a e b representam dois termos consecutivos. A variável proximo evita a perda do valor antigo de a ou b durante a atualização.

17. Fibonacci com while

Enunciado: Resolva o mesmo problema anterior, mas agora usando while em vez de for.

Solução:

```
#include <stdio.h>

int main(void) {
    int n, contador = 0;
    long long a = 0, b = 1, proximo;

    printf("Quantidade de termos: ");
    scanf("%d", &n);

    // O contador controla quantos termos já foram exibidos.
    while (contador < n) {
        printf("%lld ", a);

        proximo = a + b;
        a = b;
        b = proximo;

        contador++;
    }

    printf("\n");
    return 0;
}
```

Comentário: No while precisamos controlar explicitamente o contador. A condição contador < n determina quando o laço termina.

18. Vetor + while — cadastro até sair

Enunciado: Peça nomes ao usuário repetidamente. Guarde-os em um vetor até que ele digite “fim”. Depois, percorra e mostre todos os nomes cadastrados.

Solução:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char nomes[100][50];
    char nome[50];
    int quantidade = 0;
    int i;

    // Continua até o usuário digitar "fim" ou o vetor ficar cheio.
    while (quantidade < 100) {
        printf("Nome (ou 'fim'): ");
        scanf("%49s", nome);

        if (strcmp(nome, "fim") == 0)
            break;

        strcpy(nomes[quantidade], nome);
        quantidade++;
    }

    printf("\n\nNomes cadastrados:\n\n");

    // Percorre somente os nomes realmente cadastrados.
    for (i = 0; i < quantidade; i++)
        printf("- %s\n", nomes[i]);

    return 0;
}
```

Comentário: O break interrompe o while imediatamente quando a palavra fim é encontrada. Como C exige que o tamanho do vetor seja definido, também colocamos um limite de 100 nomes.

19. Vetor de structs — contagem de palavras

Enunciado: Leia uma frase e conte quantas vezes cada palavra aparece. Como C não possui dicionário

nativo, use um vetor de structs contendo palavra e quantidade.

Solução:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

typedef struct {
    char palavra[30];
    int quantidade;
} Contagem;

int main(void) {
    char frase[300];
    char *palavra;
    Contagem contagens[100];
    int total = 0, i, encontrada;

    printf("Digite uma frase: ");
    fgets(frase, sizeof(frase), stdin);

    // strtok separa a frase em palavras usando espaço e pontuação.
    palavra = strtok(frase, " ,.!?:;\\n");

    while (palavra != NULL) {
        encontrada = 0;

        // Procura se a palavra já foi cadastrada.
        for (i = 0; i < total; i++) {
            if (strcmp(contagens[i].palavra, palavra) == 0) {
                contagens[i].quantidade++;
                encontrada = 1;
            }
        }

        if (!encontrada) {
            strcpy(contagens[total].palavra, palavra);
            contagens[total].quantidade = 1;
            total++;
        }

        palavra = strtok(NULL, " ,.!?:;\\n");
    }

    printf("\n\nContagem de palavras:\n\n");
    for (i = 0; i < total; i++)
        printf("%s: %d\n", contagens[i].palavra, contagens[i].quantidade);

    return 0;
}
```

```

        break;
    }
}

// Se não existe, cria um novo registro.
if (!encontrada && total < 100) {
    strcpy(contagens[total].palavra, palavra);
    contagens[total].quantidade = 1;
    total++;
}

palavra = strtok(NULL, " ,.!?:;\n");
}

// Mostra a contagem final.
for (i = 0; i < total; i++)
    printf("%s: %d\n", contagens[i].palavra, contagens[i].quantidade);

return 0;
}

```

Comentário: strtok() pode dividir uma string em tokens. Como não temos um dicionário, cada struct guarda uma palavra e seu contador. O vetor de structs desempenha uma função semelhante à estrutura de contagem do exercício em Python.

20. Exercício integrado — sistema de notas

Enunciado: Faça um pequeno sistema que cadastre alunos e suas notas em um vetor de structs. O programa deve permitir cadastrar, consultar e remover alunos até que o usuário escolha sair. Ao cadastrar, armazene três notas para cada aluno e mostre a média na consulta.

Solução:

```

#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[50];
    float notas[3];
} Aluno;

int main(void) {
    Aluno alunos[100];
    int quantidade = 0;
    int opcao, i, j, encontrado;
    char nome[50];
    float media;

    do {
        printf("\n1 - Cadastrar\n");
        printf("2 - Consultar\n");
        printf("3 - Remover\n");
        printf("4 - Sair\n");
        printf("Opcao: ");
        scanf("%d", &opcao);

        if (opcao == 1) {
            if (quantidade >= 100) {
                printf("Limite de alunos atingido.\n");
                continue;
            }

            printf("Nome do aluno: ");
            scanf("%49s", alunos[quantidade].nome);

            // Lê as três notas do aluno.
            for (j = 0; j < 3; j++) {
                printf("Nota %d: ", j + 1);
                scanf("%f", &alunos[quantidade].notas[j]);
            }

            quantidade++;
        }
    } while (opcao != 4);
}

```

```

        printf("Aluno cadastrado.\n");

    } else if (opcao == 2) {
        printf("Nome do aluno: ");
        scanf("%49s", nome);
        encontrado = 0;

        // Procura o aluno pelo nome.
        for (i = 0; i < quantidade; i++) {
            if (strcmp(nome, alunos[i].nome) == 0) {
                media = (alunos[i].notas[0] +
                        alunos[i].notas[1] +
                        alunos[i].notas[2]) / 3.0f;

                printf("Notas: %.2f %.2f %.2f\n",
                        alunos[i].notas[0],
                        alunos[i].notas[1],
                        alunos[i].notas[2]);
                printf("Media: %.2f\n", media);

                encontrado = 1;
                break;
            }
        }

        if (!encontrado)
            printf("Aluno nao encontrado.\n");

    } else if (opcao == 3) {
        printf("Nome do aluno: ");
        scanf("%49s", nome);
        encontrado = 0;

        // Procura o aluno e desloca os registros seguintes.
        for (i = 0; i < quantidade; i++) {
            if (strcmp(nome, alunos[i].nome) == 0) {
                for (j = i; j < quantidade - 1; j++)
                    alunos[j] = alunos[j + 1];

                quantidade--;
                encontrado = 1;
                printf("Aluno removido.\n");
                break;
            }
        }

        if (!encontrado)
            printf("Aluno nao encontrado.\n");

    } else if (opcao == 4) {
        printf("Programa encerrado.\n");

    } else {
        printf("Opcao invalida.\n");
    }

} while (opcao != 4);

return 0;
}

```

Comentário: Este exercício reúne variáveis, entrada de dados, if/else if/else, condicionais aninhadas, while/do...while, for, vetores, strings e structs. Como C não oferece dicionários e listas dinâmicas nativamente, usamos um vetor de structs e controlamos manualmente a quantidade de alunos.